



Introducere în Programare Dinamică

Șirul lui Fibonacci · Drumuri în matrice
Subșir crescător de lungime maximă

Andrei Mihai Ciobanu

Algolymp Summer School

Ce este Programarea Dinamică?

Șirul lui Fibonacci

Drumuri în matrice

Subșir crescător de lungime maximă

Recapitulare

Ce este Programarea Dinamică?

Problemă

Se cere să se calculeze cel de-al n -lea număr Fibonacci, unde $F_1 = 1$, $F_2 = 1$, $F_n = F_{n-1} + F_{n-2}$.

Soluție recursivă naivă:

- $F(5)$ apelează $F(4)$ și $F(3)$
- $F(4)$ apelează $F(3)$ și $F(2)$
- $F(3)$ se calculează de **mai multe ori**

Complexitate: $O(2^n)$ — complet ineficient pentru n mare.

Observație cheie

Aceleași subprobleme sunt rezolvate **în mod repetat**. Dacă am memora rezultatele, am evita recalcularea.

Programare Dinamică (DP)

Paradigmă care **descompune o problemă complexă** în subprobleme mai simple, rezolvate recursiv, reținând soluțiile pentru a le reutiliza.

3 categorii de probleme DP:

1. **Numărare** — câte instanțe respectă o proprietate

Exemplu: câte drumuri există din $(1, 1)$ în (N, M)

2. **Optimizare** — maximizare sau minimizare

Exemplu: cel mai lung subșir crescător

3. **Decizie** — dacă un rezultat se poate obține

Exemplu: câștigătorul unui joc cu mișcări optime

Proprietăți DP (1): Substructura optimă

Substructura optimă

Soluția optimă a problemei **depinde de soluțiile optime** ale subproblemelor în care aceasta se descompune.

Exemplu — SCLM: fie $dp[i]$ = lungimea celui mai lung subșir crescător care se termină la poziția i . Atunci:

$$dp[i] = 1 + \max_{\substack{j < i \\ a[j] < a[i]}} dp[j]$$

Atenție

Substructura optimă nu implică neapărat DP — metodele **Greedy** și **Divide et Impera** pot fi aplicabile în unele cazuri.

Proprietăți DP (2): Subprobleme suprapuse

Subprobleme suprapuse

Dacă aceleași subprobleme apar de **mai multe ori**, o abordare naivă le recalculează în mod repetat. DP le rezolvă **o singură dată** și reține rezultatul.

Exemplu — arborele de apeluri pentru $F(5)$:

- $F(5)$ apelează $F(4)$ și $F(3)$
- $F(4)$ apelează $F(3)$ și $F(2)$
- $F(3)$ este calculat de **2 ori**; $F(2)$ de 3 ori; $F(1)$ de 5 ori ...

Soluție DP: memorăm $F(i)$ după prima calculare. Fiecare apel unic se face **o singură dată** $\Rightarrow O(2^n) \rightarrow O(n)$.

Pașii de rezolvare

1. **Identificăm** că problema poate fi rezolvată cu DP
2. **Definim stările** — cum descriem o subproblemă
ex: $dp[i]$ = al i -lea Fibonacci, $dp[l][r]$ = soluția pe intervalul $[l, r]$
3. **Identificăm cazurile de bază** — stările simple cu răspuns direct
4. **Găsim relația de recurență** (tranziția)
5. **Alegem structura de date** — vector, matrice, hashmap
6. **Stabilim ordinea de calcul** — **bottom-up** (iterativ, de la stări mici) sau **top-down** (recursiv cu memoizare)
7. Dacă am ajuns până aici fără probleme — *Voilà!*, problema e rezolvată

Memoizare vs. Tabelarizare

Top-down (memoizare)

- Implementare **recursivă**
- + Derivă direct din recurență
- + Calculează doar stările necesare
- Risc de stack overflow la n mare

Bottom-up (tabelarizare)

- Implementare **iterativă**
- + Fără recursivitate, mai rapid
- + Control complet al ordinii
- Necesită ordine de calcul explicită

Recomandare: bottom-up e preferat în competiții — mai rapid și fără risc de stack overflow.

Șirul lui Fibonacci

Fibonacci — Abordare DP

Aplicăm pașii de rezolvare la Fibonacci:

1. **Stare:** $dp[i]$ = valoarea F_i
2. **Bază:** $dp[1] = dp[2] = 1$
3. **Recurență:** $dp[i] = dp[i - 1] + dp[i - 2]$
4. **Structură:** vector de n elemente
5. **Ordine:** de la $i = 3$ la n (bottom-up)

Primele valori

i	1	2	3	4	5	6
$dp[i]$	1	1	2	3	5	8

Fibonacci — Top-down (memoizare)

```
1  const int nmax = 100;
2  int memo[nmax + 1];
3
4  int fib(int n) {
5      if (n <= 2) {
6          return 1;
7      }
8      if (memo[n] != 0) {
9          return memo[n];
10     }
11     memo[n] = fib(n - 1) + fib(n - 2);
12     return memo[n];
13 }
```

Complexitate: $O(n)$ timp, $O(n)$ spațiu.

Fibonacci — Bottom-up (tabelarizare)

```
1  const int nmax = 100;
2  int dp[nmax + 1];
3
4  void calcFib(int n) {
5      dp[1] = 1;
6      dp[2] = 1;
7      for (int i = 3; i <= n; i++) {
8          dp[i] = dp[i - 1] + dp[i - 2];
9      }
10 }
```

Tipar DP

Stare: $dp[i]$ = al i -lea număr Fibonacci

Tranziție: $dp[i] = dp[i - 1] + dp[i - 2]$

Bază: $dp[1] = dp[2] = 1$

Drumuri în matrice

Numărarea drumurilor pe grid

Problemă

Se dă o matrice $n \times m$ cu valori 0 (liber) și 1 (blocat). Câte drumuri există de la $(1, 1)$ la (n, m) , mergând doar **dreapta** sau **jos**?

Observație: pentru a ajunge în (i, j) , trebuie să fi venit din $(i - 1, j)$ sau $(i, j - 1)$.

Recurență

$$dp[i][j] = \begin{cases} 0 & \text{dacă } a[i][j] = 1 \text{ (blocat)} \\ dp[i - 1][j] + dp[i][j - 1] & \text{altfel} \end{cases}$$

Bază: $dp[1][1] = 1$ (dacă celula $(1, 1)$ e liberă).

Drumuri pe grid — Exemplu pas cu pas

Matrice 3×3 , toate celulele libere:

Grila (0 = liber):

0	0	0
0	0	0
0	0	0

Valorile $dp[i][j]$:

1	1	1
1	2	3
1	3	6

Există **6** drumuri din (1, 1) în (3, 3). **Ordine de calcul:** rând cu rând, stânga la dreapta — garantează că $dp[i-1][j]$ și $dp[i][j-1]$ sunt deja calculate.

DP pe matrice — Implementare

```
1  const int nmax = 1000;
2  long long dp[nmax + 1][nmax + 1] = {};
3  // a[][], n, m citite anterior
4  dp[1][1] = 1;
5
6  for (int i = 1; i <= n; i++) {
7      for (int j = 1; j <= m; j++) {
8          if (a[i][j] == 1 || (i == 1 && j == 1)) {
9              continue;
10         }
11         dp[i][j] = dp[i - 1][j] + dp[i][j - 1];
12     }
13 }
14 // Raspuns: dp[n][m]
```

Complexitate: $O(n \cdot m)$ timp și spațiu.

**Subșir crescător de lungime
maximă**

Problemă

Dat un șir $a_1, a_2, \dots, a_n$, găsește lungimea celui mai lung **subșir strict crescător**.

Exemplu: $a = [3, 1, 4, 1, 5, 9, 2, 6]$

- Un subșir crescător: 1, 4, 5, 9 — lungime 4
- Un alt subșir: 1, 2, 6 — lungime 3
- Răspuns: 4

Atenție la termeni

Subșir (subsequence) $\neq$ **subșir contiguu** (subarray). Elementele nu trebuie să fie consecutive — doar în ordine.

SCLM — Cum gândim soluția

Stare: $dp[i]$ = lungimea celui mai lung subșir crescător care se termină la poziția i . De ce fixăm ultimul element? Altfel nu știm cu ce elemente putem extinde subșirul în continuare. **Cum calculăm $dp[i]$?** Avem de ales:

1. Subșirul are doar elementul i : $dp[i] = 1$
2. Extindem un subșir anterior: căutăm $j < i$ cu $a[j] < a[i]$ și luăm $dp[i] = dp[j] + 1$

Vrem maximul, deci:

$$dp[i] = 1 + \max_{\substack{j < i \\ a[j] < a[i]}} dp[j]$$

Răspuns: $\max_{i=1}^n dp[i]$

SCLM — Exemplu pas cu pas

$$a = [3, 1, 4, 1, 5, 9, 2, 6]$$

i	1	2	3	4	5	6	7	8
$a[i]$	3	1	4	1	5	9	2	6
$dp[i]$	1	1	2	1	3	4	2	4

Detaliu pentru $i = 5$ ($a[5] = 5$):

- $j = 1$: $a[1] = 3 < 5 \Rightarrow$ candidat $dp[1] + 1 = 2$
- $j = 3$: $a[3] = 4 < 5 \Rightarrow$ candidat $dp[3] + 1 = 3$
- $dp[5] = 3$ — subșirul $[1, 4, 5]$ sau $[3, 4, 5]$

SCLM — Implementare

```
1  const int nmax = 100000;
2  int a[nmax + 1], dp[nmax + 1];
3
4  // n, a[] citite anterior
5  int ans = 1;
6  for (int i = 1; i <= n; i++) {
7      dp[i] = 1;
8      for (int j = 1; j < i; j++) {
9          if (a[j] < a[i]) {
10             dp[i] = max(dp[i], dp[j] + 1);
11         }
12     }
13     ans = max(ans, dp[i]);
14 }
15 // ans = lungimea SCLM
```

Complexitate: $O(n^2)$ timp, $O(n)$ spațiu.

▷ Există și o soluție $O(n \log n)$ cu sortare și căutare binară — pentru cei curioși.

Recapitulare

Recapitulare

Problemă	Stare	Complexitate
Fibonacci	$dp[i]$	$O(n)$
Drumuri pe grid	$dp[i][j]$	$O(n \cdot m)$
SCLM	$dp[i] = \text{lung. max ce se termină la } i$	$O(n^2)$

Tiparul DP:

1. Definești **starea** $dp[i]$ (sau $dp[i][j]$)
2. Scrii **recurența** (tranziția)
3. Stabilești **cazul de bază**
4. Calculezi în ordine corectă (**bottom-up**)

Alte tipuri de Programare Dinamică

Lecțiile de azi au acoperit DP **pe un șir** și **pe o matrice**. Există mult mai multe variante:

- **DP pe două șiruri** — Subșir comun maximal (lecția următoare)
- **DP cu rucsac** — Problema Rucsacului (lecția următoare)
- **Range DP** — pe intervale $[l, r]$
- **Digit DP** — pe cifrele unui număr
- **Bitmask DP** — starea e o mască de biți
- **DP pe DAG / arbore** — pe structuri de graf (lot)

Probleme de antrenament

1. **lepuras** (pbinfo #356)
2. **Poduri** (pbinfo #22)
3. **LSC** (pbinfo #80 — SCLM)
4. **Array Description** (CSES #1746)
5. **Removing Digits** (CSES #1637)
6. **Increasing Subsequence** (CSES #1145 — SCLM)

1. **pbinfo** — Programare Dinamică (în română)
2. **USACO Guide** — Introduction to DP (Gold)
3. **CP-Algorithms** — Introduction to Dynamic Programming